Kamily Hamud
Carolina Greiffo

PAM - Privileged Access Management

CURITIBA

2025

1.Introdução

Este trabalho implementa uma arquitetura PAM (Privileged Access Management) usando certificados digitais temporários para autenticar conexões SSH. O sistema roda com três elementos principais, todos em containers Docker, que trocam informações numa rede própria.

O foco é garantir acesso seguro a servidores privilegiados, e para isso foi implementado:

- Autenticação MFA;
- Certificados digitais com validade de 10 minutos;
- Autoridade Certificadora (CA);
- Eliminação completa do uso de senha.

2. Arquitetura do Sistema

De início, foi criado o diretório “pam-project” e a estrutura de pastas para organizar os três subdiretórios do sistema. Assim, cada subdiretório tem seus próprios arquivos de configuração e Dockerfiles, facilitando a manutenção.

2.2. Criação da rede docker

Foi criada uma rede docker privada para comunicação entre os containers. Essa rede permite que os três containers se comuniquem entre si de forma segura.

```
kamily@ubuntu:~/Área de trabalho$ cd
kamily@ubuntu:~$ mkdir ~/pam-project
kamily@ubuntu:~$ cd ~/pam-project
kamily@ubuntu:~/pam-project$ mkdir -p vault/data
kamily@ubuntu:~/pam-project$ mkdir -p vault/data
kamily@ubuntu:~/pam-project$ mkdir -p ssh-server
kamily@ubuntu:~/pam-project$ mkdir -p signer
kamily@ubuntu:~/pam-project$ ls
signer  ssh-server  vault
kamily@ubuntu:~/pam-project$ docker network create pam-network
264b94b75189f0dca3092b72ab263e824f0f38cae585802e7a2ad519ae30919a
kamily@ubuntu:~/pam-project$ cd ~/pam-project/vault
kamily@ubuntu:~/pam-project/vault$ nano Dockerfile
kamily@ubuntu:~/pam-project/vault$ nano entrypoint.sh
kamily@ubuntu:~/pam-project/vault$ chmod +x entrypoint.sh
kamily@ubuntu:~/pam-project/vault$ docker build -t pam-vault .
[+] Building 39.7s (10/10) FINISHED                          docker:default
 => [internal] load build definition from Dockerfile                    0.0s
 => => transferring dockerfile: 260B                                    0.0s
 => [internal] load metadata for docker.io/library/ubuntu:22.04        13.0s
 => [internal] load .dockerignore                                       0.0s
 => => transferring context: 2B                                         0.0s
 => [1/5] FROM docker.io/library/ubuntu:22.04@sha256:09506232a8004baa32c4  4.5s
 => => resolve docker.io/library/ubuntu:22.04@sha256:09506232a8004baa32c4  0.0s
 => => sha256:09506232a8004baa32c47d68f1e5c307d648fdd59f5 6.69kB / 6.69kB  0.0s
 => => sha256:4cb780d50443fc4463f1f9360c03ca46512e4fdd8fd97c5 424B / 424B  0.0s
 => => sha256:392fa14dddd09da29a5c3d26948ff81c494424035b7 2.30kB / 2.30kB  0.0s
 => => sha256:af6eca94c8104c8e90d3f9efe59c2b3a02b20aad3 29.54MB / 29.54MB  2.0s
 => => extracting sha256:af6eca94c8104c8e90d3f9efe59c2b3a02b20aad3d985e31  2.2s
 => [internal] load build context                                       0.0s
 => => transferring context: 348B                                       0.0s
 => [2/5] RUN apt-get update && apt-get install -y     openssh-client     21.2s
```

3.Implementação do Vault (CA)

Foram criados dois arquivos importantes no diretório vault/, o arquivo Dockerfile e o entrypoint.sh. Dockerfile, define a imagem base do sistema operacional e instala o pacote openssh-client, que contém o comando necessário para gerar e assinar os certificados, também define que o diretório de trabalho será "/vault" e copia um script de inicialização chamado entrypoint.sh para dentro da imagem, para poder ter permissões de execução. Enquanto o entrypoint faz a verificação do par de chaves da CA e caso o par não exista, o script executa o comando ssh-keygen com parâmetros específicos: tipo RSA, tamanho de 4096 bits, sem senha de proteção e com um comentário identificador. Este comando gera dois arquivos: ca_key contendo a chave privada e ca_key.pub contendo a chave pública.

```
kamily@ubuntu:~/pam-project/vault$ nano Dockerfile
kamily@ubuntu:~/pam-project/vault$ nano entrypoint.sh
```

```
sshuser@a221bff0372a: ~
kamily@ubuntu:~$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/pam_key -N ""
Generating public/private rsa key pair.
/home/kamily/.ssh/pam_key already exists.
Overwrite (y/n)? y
Your identification has been saved in /home/kamily/.ssh/pam_key
Your public key has been saved in /home/kamily/.ssh/pam_key.pub
The key fingerprint is:
SHA256:t6hlNZp7LDGvU46DVk8YfqTBY0F8lrmQfGehYMrfy78 kamily@ubuntu
The key's randomart image is:
+---[RSA 4096]----+
|        ++. o..   |
|      . o*.*.o    |
|       o. *.+     |
|        .*.o      |
|        oSB=      |
|         BB++     |
|        o*%+      |
|       o+=.*.     |
|      .. o*  E.   |
+----[SHA256]-----+
```

Com os arquivos prontos, foi executado o comando docker build para construir a imagem, seguido do docker run para iniciar o container. Durante a execução, foi especificado que o container deveria rodar em modo detached (background), conectar-se à rede pam-network e montar um volume que mapeia o diretório vault/data do host para o diretório /vault dentro do container.

4.Implementação do servidor SSH

O SSH-server é responsável por aceitar conexões SSH autenticadas exclusivamente através de certificados digitais. Esta implementação envolveu a criação de três arquivos no diretório ssh-server.

O Dockerfile do SSH-server também utiliza o sistema operacional como base, instala o pacote openssh-server ao invés do client. Durante a construção da imagem, o script cria o diretório /var/run/sshd que é necessário para o funcionamento do SSH e então cria um usuário do sistema chamado "sshuser" com seu diretório home. As permissões do diretório .ssh são configuradas garantindo que apenas o proprietário possa acessar. O Dockerfile também copia os arquivos de configuração importantes: sshd_config e setup.sh, abre a porta 22 e define o setup.sh como comando de inicialização.

O arquivo sshd_config contém as configurações de segurança do servidor SSH. As configurações mais importantes são desligar completamente a autenticação por senha através da linha PasswordAuthentication no, e a configuração TrustedUserCAKeys apontando para /etc/ssh/ca_key.pub. Esta última linha é de extrema importancia para este sistema PAM criado, pois instrui o servidor SSH a confiar em certificados digitais que foram assinados pela chave privada correspondente à chave pública especificada.

```
kamily@ubuntu:~/pam-project/ssh-server$ nano Dockerfile
kamily@ubuntu:~/pam-project/ssh-server$ nano sshd_config
```

O script setup.sh cria uma sincronização com o Vault. Ao iniciar, o script entra em um loop que verifica repetidamente se a chave pública da CA já foi gerada pelo container Vault. Este loop só para quando o arquivo ca_key.pub está disponível na pasta compartilhada. Quando encontrado, o script copia esta chave pública para o local onde o SSH espera encontrá-la. Por fim, o script inicia o servidor SSH mantendo ele rodando continuamente.

A execução do SSH Server foi configurada para acessar a pasta do Vault apenas para leitura, garantindo que o servidor SSH possa ver a chave pública da CA mas não possa alterá-la. A porta 22 do contêiner foi ligada à porta 2222 do computador, permitindo que usuários se conectem ao servidor através desta porta sem gerar conflito com um possível SSH Server já rodando no computador principal.

```
kamily@ubuntu:~/pam-project/ssh-server$ nano setup.sh
kamily@ubuntu:~/pam-project/ssh-server$ chmod +x setup.sh
kamily@ubuntu:~/pam-project/ssh-server$ docker build -t pam-ssh-server .
[+] Building 46.0s (12/12) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                            0.0s
 => => transferring dockerfile: 538B                                            0.0s
 => [internal] load metadata for docker.io/library/ubuntu:22.04                 7.9s
 => [internal] load .dockerignore                                               0.0s
 => => transferring context: 2B                                                 0.0s
 => CACHED [1/7] FROM docker.io/library/ubuntu:22.04@sha256:09506232a8004baa32c4  0.0s
 => [internal] load build context                                               0.0s
 => => transferring context: 600B                                               0.0s
 => [2/7] RUN apt-get update && apt-get install -y     openssh-server     sudo   34.5s
 => [3/7] RUN mkdir /var/run/sshd                                               0.4s
 => [4/7] RUN useradd -m -s /bin/bash sshuser &&     echo "sshuser:password123"   1.2s
 => [5/7] COPY sshd_config /etc/ssh/sshd_config                                 0.1s
 => [6/7] COPY setup.sh /setup.sh                                               0.0s
 => [7/7] RUN chmod +x /setup.sh                                                0.5s
 => exporting to image                                                          1.2s
 => => exporting layers                                                         1.2s
 => => writing image sha256:53b097f0fb1f51a0e121d7adae2b53aea263343157548858241f  0.0s
 => => naming to docker.io/library/pam-ssh-server                               0.0s
kamily@ubuntu:~/pam-project/ssh-server$ docker run -d --name ssh-server --network pam-
etwork \
>  -v ~/pam-project/vault/data:/ca-keys:ro \
> -p 2222:22 \
> pam-ssh-server
a221bff0372af9af042b0b4c15cabf3c758365de706c1827e95be25c079a0400
kamily@ubuntu:~/pam-project/ssh-server$
```

4.Implementação da Interface Web - Signer

O terceiro e último a ser criado foi o Signer, uma aplicação web feita em python usando o framework Flask, que serve como interface entre os usuários e o sistema PAM, permitindo configurar a autenticação de dois fatores e solicitar certificados digitais.

Começou com o arquivo requeriments.txt, com todas as bibliotecas python necessárias. A biblioteca Flask foi escolhida para criar o servidor web e as APIs. A biblioteca pyotp foi incluída para fazer o algoritmo TOTP (time-based One-Time Password) usado na autenticação de dois fatores. As bibliotecas QRcode e pillow foram usadas para permitir criar códigos QR que podem ser lidos por aplicativos autenticadores como o Google Authenticator.

O arquivo principal app.py foi criado contendo toda a lógica de aplicação. A aplicação Flask foi configurada para guardar os segredos MFA dos usuários em um dicionário python na memória. Na realidade, esse método não é seguro e nem indicado, pois os segredos deveriam ser guardados em um banco de dados criptografado.

```
kamily@ubuntu:~/pam-project/ssh-server$ cd ~/pam-project/signer
kamily@ubuntu:~/pam-project/signer$ nano requirements.txt
kamily@ubuntu:~/pam-project/signer$ nano app.py
kamily@ubuntu:~/pam-project/signer$
```

```
kamily@ubuntu:~/pam-project/signer$ nano Dockerfile
kamily@ubuntu:~/pam-project/signer$ docker build -t pam-signer .
[+] Building 35.8s (6/10)                                          docker:default
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 300B                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim          12.4s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [1/6] FROM docker.io/library/python:3.11-slim@sha256:fa9b525a0be0c5ae5e6f220  6.3s
 => => resolve docker.io/library/python:3.11-slim@sha256:fa9b525a0be0c5ae5e6f220  0.0s
 => => sha256:870925f757415a696459255b0a20b082e914cac42efaff4424 5.48kB / 5.48kB  0.0s
 => => sha256:d7ecded7702a5dbf6d0f79a71edc34b534d08f3051980e2c 29.78MB / 29.78MB  1.9s
 => => sha256:1ee9c106547f05aa380c4cdec2837c546439943d73d965a3fc 1.29MB / 1.29MB  1.1s
 => => sha256:f002d17b63fe84a7f8a66f20cfa63aec4f6cd2a44069f05b 14.36MB / 14.36MB  2.9s
 => => sha256:fa9b525a0be0c5ae5e6f2209f4be6fdc5a15a36fed022214 10.37kB / 10.37kB  0.0s
 => => sha256:b596083aa14d47c78a652138aa9b98607585499d7c7ec343ae 1.75kB / 1.75kB  0.0s
 => => sha256:65868b001a40155a1d3f5aa7f5a10ba02a7d55697301839dc047c9 248B / 248B  1.8s
 => => extracting sha256:d7ecded7702a5dbf6d0f79a71edc34b534d08f3051980e2c948fba7  2.1s
 => => extracting sha256:1ee9c106547f05aa380c4cdec2837c546439943d73d965a3fc49f22  0.2s
 => => extracting sha256:f002d17b63fe84a7f8a66f20cfa63aec4f6cd2a44069f05b6296b0a  1.9s
 => => extracting sha256:65868b001a40155a1d3f5aa7f5a10ba02a7d55697301839dc047c9d  0.0s
 => [internal] load build context                                             0.0s
 => => transferring context: 13.97kB                                          0.0s
 => [2/6] WORKDIR /app                                                        0.4s
 => [3/6] RUN apt-get update &&     apt-get install -y docker.io &&     rm -rf   16.6s
 => => # _64-linux-gnu/perl/5.40 /usr/share/perl/5.40 /usr/local/lib/site_perl) at /us
 => => # r/share/perl5/Debconf/FrontEnd/Readline.pm line 8, <STDIN> line 123.)
 => => # debconf: falling back to frontend: Teletype
 => => # debconf: unable to initialize frontend: Teletype
 => => # debconf: (This frontend requires a controlling tty.)
 => => # debconf: falling back to frontend: Noninteractive
```

A interface HTML e CSS foi feita diretamente no arquivo python para evitar muitos arquivos e problemas de conexão. A interface foi dividida em três sessões, cada uma representando um passo do processo: gerar par de chaves, configurar MFA e certificação digital CA, podendo ser acessada em http://localhost:5000/. Em seguida, a explicação do passo a passo.

Passo 1 - Geração do par de chaves SSH. O usuário executa o comando ssh-keygen no terminal para gerar duas chaves: a chave privada que fica guardada no computador e a chave pública que será enviada para o sistema. O comando cat exibe o conteúdo da chave pública para o usuário copiar.

Passo 2 - Configuração de Autenticação MFA. A primeira função criada foi a função principal que mostra a página HTML quando o usuário acessa a aplicação. A segunda função, /setup-mfa, recebe o nome de usuário e cria um código secreto usando a biblioteca pyotp. Este código fica guardado junto com o nome do usuário. Depois, a função cria um QR code com este código secreto e manda para o navegador mostrar na tela. O usuário pode ler o QR code com o celular através do Google Authenticator.

Passo 3 - Requisição de Certificado Digital. A terceira função, /request-certificate, é a mais importante e cuida de validar o código MFA e organizar a assinatura do certificado. Esta função recebe três coisas: nome do usuário, código MFA de seis dígitos e a chave pública SSH. Primeiro ela verifica se todos os campos foram preenchidos e se o usuário já configurou o MFA antes. Depois ela pega o código secreto guardado do usuário e verifica se o código de seis dígitos está correto. A verificação aceita códigos gerados até 30 segundos antes. Se o código MFA estiver correto, começa o processo de assinatura do certificado. A chave pública do usuário é salva temporariamente em um arquivo. Usando comandos docker, o código copia este arquivo para dentro do container Vault. Depois executa o comando ssh-keygen dentro do Vault para assinar o certificado. Este comando usa a chave privada da CA para assinar, define o nome do usuário no certificado como "sshuser" e configura a validade de 10 minutos. O certificado pronto é lido de volta do Vault e enviado como resposta para o usuário.

O Dockerfile do Signer usa python e instala o Docker CLI dentro do container. Esta instalação permite que o Signer execute comandos docker para se comunicar com os outros containers. O socket do docker do computador é compartilhado com o container Signer, permitindo que ele controle outros containers do sistema.

```
kamily@ubuntu:~/pam-project/signer$ docker run -d --name signer --network pam-network \
>  -p 5000:5000 \
>  -v /var/run/docker.sock:/var/run/docker.sock \
>  pam-signer
b66e236783546d05aface7c440547b6290bc8935d5baab6266450bb41fe2ead8
kamily@ubuntu:~/pam-project/signer$
```

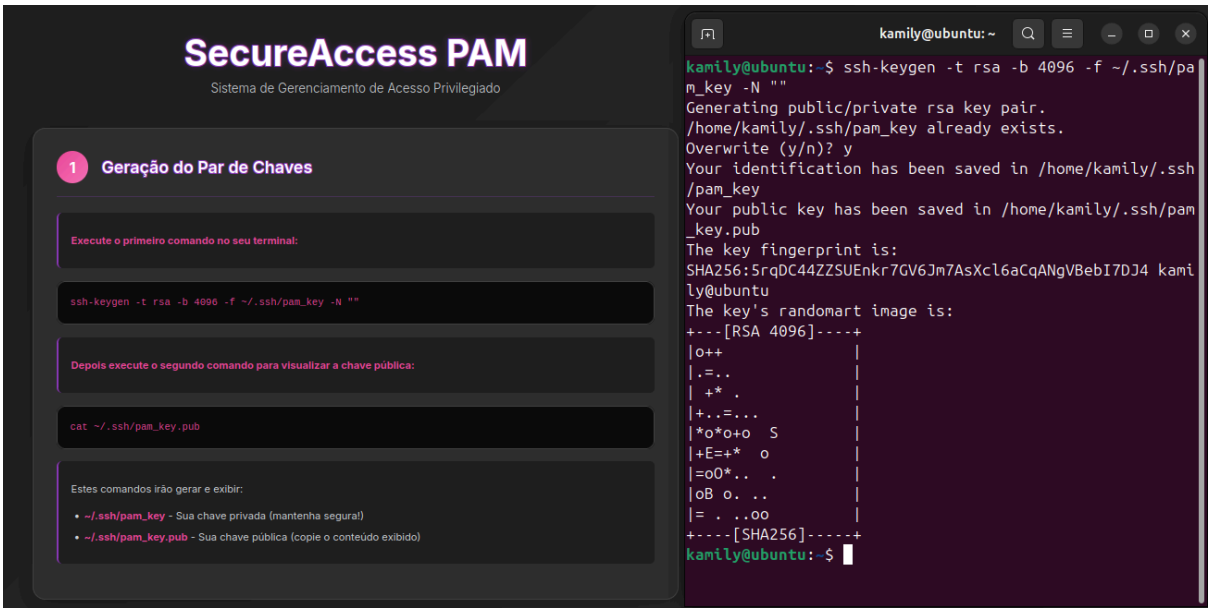
Com os três dockers criados em execução, o sistema foi verificado através do comando docker ps para confirmar que todos os contêineres estavam rodando corretamente e conectados à rede pam-network.

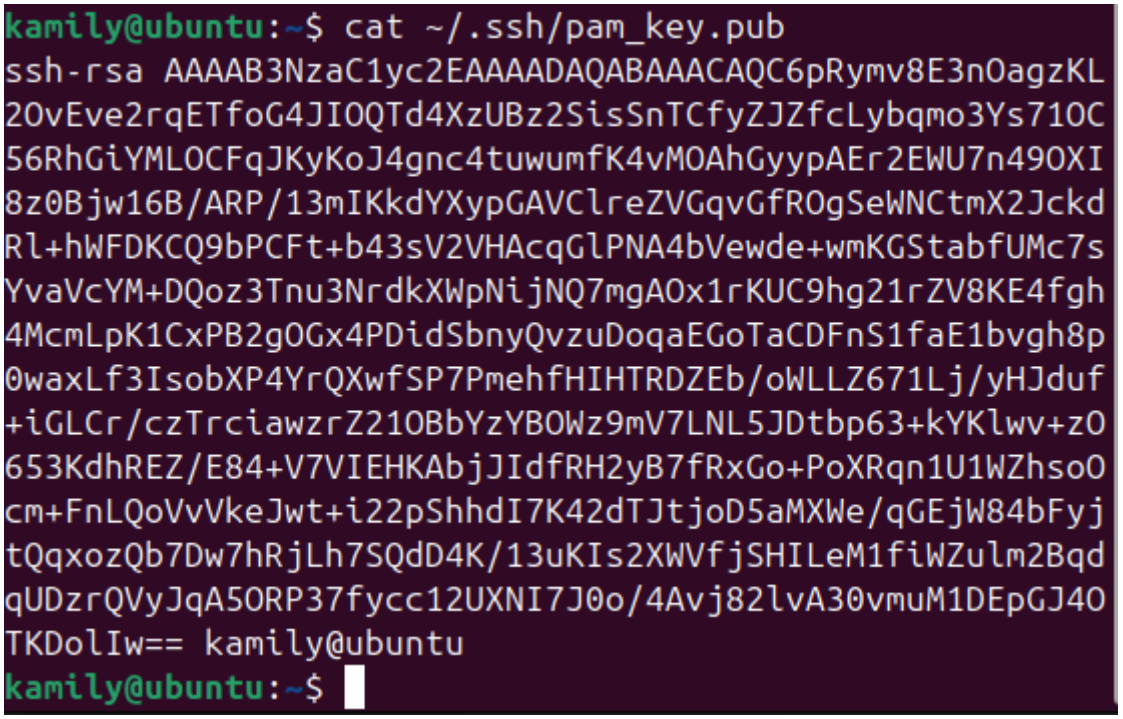
```
kamily@ubuntu:~/pam-project/signer$ docker ps
CONTAINER ID   IMAGE            COMMAND              CREATED          STATUS          POR
TS                                                 NAMES
b66e23678354   pam-signer       "python app.py"      2 minutes ago    Up 2 minutes    0.0
.0.0:5000->5000/tcp, [::]:5000->5000/tcp   signer
a221bff0372a   pam-ssh-server   "/setup.sh"          23 minutes ago   Up 23 minutes   0.0
.0.0:2222->22/tcp, [::]:2222->22/tcp       ssh-server
7e95b5658198   pam-vault        "/entrypoint.sh"     44 minutes ago   Up 44 minutes
                                                   vault
```

5.Demonstração visual do funcionamento do projeto PAM

A interface web foi testada acessando o endereço localhost:5000 pelo navegador, e foi confirmado que a página exibia corretamente os três passos iniciais: geração do par de chaves, configuração via MFA com QR code e envio da chave pública para gerar o certificado com validade de 10 minutos. Após a conclusão destes passos, a página se estende para o passo quatro, que consiste em salvar o certificado, e por

fim o passo cinco: realizar a conexão via SSH. Todo o fluxo do sistema foi validado, começando pela geração do par de chaves pública/privada, seguido da configuração do MFA com leitura do QR code e então a geração do certificado contendo o nome do usuário, a chave e o código de autenticação. Por fim, o certificado recebido foi utilizado para realizar o login no servidor SSH. Todos os elementos foram feitos para funcionar de forma integrada, estabelecendo um sistema em que o usuário precisa passar por autenticação de dois fatores para obter um certificado temporário que permite a conexão ao servidor SSH por 10 minutos. Como pode ser visto nas imagens a seguir:





2 Autenticação via MFA

Configure a autenticação de dois fatores para seu usuário.

Carolina

Gerar QR Code MFA

MFA Configurado!

Escaneie o QR Code com Google Authenticator ou Authy:

Ou insira este código manualmente:

5VSIXFYNRCVGLLISIWTW2YAZY25BBLN3

3 Envio da Chave Pública

Carolina

623808

czTrciawzrZ21OBbYzYBOWz9mV7LNL5JDtbp63+kYKlwv+zO653KdhREZ/
E84+V7VIEHKAbjJIdfRH2yB7fRxGo+PoXRqn1U1WZhsoOcm+FnLQoVvVkeJwt+i22pShhdI7K42dTJtjoD5aMXWe/
qGEjW84bFyjtQqxozQb7Dw7hRjLh7SQdD4K/13uKIs2XWVfjSHILeM1fiWZulm2BqdqUDzrQVyJqA5ORP37fycc12UXN
I7J0o/4Avj82lvA30vmuM1DEpGJ4OTKDolIw== kamily@ubuntu

Solicitar Certificado

Certificado Gerado com Sucesso!

Validade: 10 minutos

Passo 4: Salvar o certificado

Execute este comando para abrir o editor:

```
cat > ~/.ssh/pam_key-cert.pub << 'EOF'
```

Depois cole o certificado abaixo:


```
FH5ujmLlTa6wUqUQo5iDvIpQ1aekVlY0TWOY6UEIg0ZEi2n2HLzwF+zG6pe/
izRrvxJJix0kyBnCz6npiO7ySHleqr4IpvUjimSRZZltJHkPhSmIc5jzGpJcOmwpHElcanTS2r1jgspn1t6BkZRYdXm+T+xeVea
u+lz//
aHdZqmD68T9Bd7QKT189SVIQFbhK4TjXcpWsy0Z47JgkdlIBOmt+RhaoDfkeEx4IwEtG2NN7/0LU3A5Xz/6KaKMrK4iIZD/8WVL
kamily@ubuntu
```


Por fim, digite EOF e pressione Enter para finalizar:

```
EOF
```

Passo 5: Conectar via SSH

```
ssh -i ~/.ssh/pam_key -o IdentitiesOnly=yes -o PreferredAuthentications=publickey -p 2222
sshuser@localhost
```

Atenção: Este certificado expira em 10 minutos. Após esse período, você precisará solicitar um novo.

```
kamily@ubuntu:~$ cat > ~/.ssh/pam_key-cert.pub << 'EOF'
>
ssh-rsa-cert-v01@openssh.com AAAAHHNzaC1yc2EtY2VydC12MD
FAb3BlbnNzaC5jb20AAAAgZqt0EiybpI8c7tIgSTH8oQ+zGNav+q6Z4
nkcyAbXkVsAAAADAQABAAACAQC6pRymv8E3nOagzKL2OvEve2rqETfo
G4JIOQTd4XzUBz2SisSnTCfyZJZfcLybqmo3Ys71OC56RhGiYMLOCFq
JKyKoJ4gnc4tuwumfK4vMOAhGyypAEr2EWU7n49OXI8z0Bjw16B/ARP
/13mIKkdYXypGAVClreZVGqvGfROgSeWNCtmX2JckdRl+hWFDKCQ9bP
CFt+b43sV2VHAcqGlPNA4bVewde+wmKGStabfUMc7sYvaVcYM+DQoz3
Tnu3NrdkXWpNijNQ7mgAOx1rKUC9hg21rZV8KE4fgh4McmLpK1CxPB2
gOGx4PDidSbnyQvzuDoqaEGoTaCDFnS1faE1bvgh8p0waxLf3IsobXP
4YrQXwfSP7PmehfHIHTRDZEb/oWLLZ671Lj/yHJduf+iGLCr/czTrci
awzrZ21OBbYzYBOWz9mV7LNL5JDtbp63+kYKlwv+zO653KdhREZ/E84
+V7VIEHKAbjJIdfRH2yB7fRxGo+PoXRqn1U1WZhsoOcm+FnLQoVvVke
Jwt+i22pShhdI7K42dTJtjoD5aMXWe/qGEjW84bFyjtQqxozQb7Dw7h
RjLh7SQdD4K/13uKIs2XWVfjSHILeM1fiWZulm2BqdqUDzrQVyJqA5O
RP37fycc12UXNI7J0o/4Avj82lvA30vmuM1DEpGJ4OTKDolIwAAAAAA
AAAAAAAAAQAAAAhDYXJvbGluYQAAAAsAAAAHc3NodXNlcgAAAABpEMS
UAAAAAGkQx1AAAAAAAAAAggAAABVwZXJtaXQtWDExLWZvcndhcmRpbm
cAAAAAAAAAF3Blcm1pdC1hZ2VudC1mb3J3YXJkaW5nAAAAAAAAABZwZ
XJtaXQtcG9ydC1mb3J3YXJkaW5nAAAAAAAAAApwZXJtaXQtcHR5AAAA
AAAAAA5wZXJtaXQtdXNlci1yYwAAAAAAAAAAAAACFwAAAAdzc2gtcnN
hAAAAAwEAAQAAAgEApXub77tHupP+gaOOcyD7HcsymhIUVdMMgUPMcx
NDUIlvUEuKiHg9wFzSD40IV0f9+c3qPXN9VO2SGpmnFFSnTORVH5Ow+
xUFZ1aP2dTlcEE9bjD+CNUVSvJrxDoZun4T6+rIh/hG18Xep9MM/iLu
a8GEGeIgsKnowb1oeFHemhg7qox0xw6+l0uXETR1EnHHq4ZjxHtJjxN
w7lyNynJ1zfUCcfRr2Z8iFFLEIJbQ9j3+Wo5OS4Q/FV4Z0oXZHtxjPD
1pHC3VhG8BGgR5x79EQML1FSGj/cay+BOeQpJhuq0cdA23YKsiCon1C
Y5x0mW981DwQC+1L4ovmNysX4Sg62oUNfCv7xVNAiC5WnoizP1oZp90
tFXBHbFnXznToDgAkK3O7Ofzb5VSKbzFDsg+rH4aEqV7a+bGNpX4sbt
bwWHlTI4K3Jf1jZK7ib7JfLuWnOi0xbBSVtuBVoW/xfhS6RgE46svHx
C8mjcSKtcyhyqATS7N5qxhe9sO/NUPrVkU/Dl63AK4i2qYD8CpaR5ld
n6hT85lJoLfby1l+P20YO4wO8P0hyZE/e99G0phajYYr8yweeYjQUfC
zezYWAv5r0B6UYjW4s/Pisk9hZCQESxjIFjTgY8D2NEFhvjBMmXdXkj
```

```
kamily@ubuntu:~$ ssh -i ~/.ssh/pam_key -o IdentitiesOnl
y=yes -o PreferredAuthentications=publickey -p 2222 ssh
user@localhost
Last login: Sun Nov  9 16:39:11 2025 from 172.18.0.1
sshuser@a221bff0372a:~$
```

```
kamily@ubuntu:~$ ls -la ~/.ssh/pam_key*
-rw------- 1 kamily kamily 3381 nov  9 13:42 /home/kamily/.ssh/pam_key
-rw-rw-r-- 1 kamily kamily 2533 nov  9 13:46 /home/kamily/.ssh/pam_key-cert.pub
-rw-r--r-- 1 kamily kamily  739 nov  9 13:42 /home/kamily/.ssh/pam_key.pub
kamily@ubuntu:~$ ssh-keygen -L -f ~/.ssh/pam_key-cert.pub
/home/kamily/.ssh/pam_key-cert.pub:1: invalid key: invalid format
/home/kamily/.ssh/pam_key-cert.pub:2:
        Type: ssh-rsa-cert-v01@openssh.com user certificate
        Public key: RSA-CERT SHA256:5rqDC44ZZSUEnkr7GV6Jm7AsXcl6aCqANgVBebI7DJ4
        Signing CA: RSA SHA256:zhzKW1r7QklJWGURgV4FJwuQyPk6CqGvEbyfNVd0Xr8 (usi
ng rsa-sha2-512)
        Key ID: "Carolina"
        Serial: 0
        Valid: from 2025-11-09T13:43:00 to 2025-11-09T13:54:40
        Principals:
                sshuser
        Critical Options: (none)
        Extensions:
                permit-X11-forwarding
                permit-agent-forwarding
                permit-port-forwarding
                permit-pty
                permit-user-rc
kamily@ubuntu:~$
```

Conclusão

Este trabalho apresenta um sistema de controle de acessos privilegiados (PAM) que usa certificados digitais temporários e autenticação em duas etapas. A estrutura foi feita com containers Docker, que isolam e conectam os elementos: Vault, SSH-server e Signer. O sistema elimina o uso de senhas e permite acesso via SSH por apenas 10 minutos.